Linguaggi di Programmazione

Semantica Dinamica: approccio operazionale.

Roberto Gorrieri

Alma Mater Studiorum - Università di Bologna

Primer di Semantica Operazionale Strutturata (SOS)

Un mini-linguaggio di programmazione

- **Linguaggio**
  - definito tramite sintassi astratta (semplice, intuitiva ma ambigua)
  - una stringa viene sempre accoppiata ad un albero sintattico (non ambiguo)
- **Insiemi di base**
  - booleani: $\{tt, ff\}$ con metavariabili $t, t_1, t' \in \mathbb{T}$
  - numeri naturali: $\{0, 1, 2, ...\}$ con metavariabili $n, m, p \in \mathbb{N}$
  - variabili: $\{a, b, c, ..., z\}$ (in numero finito) con metavariabili $v \in Var$
- **Insiemi derivati** (in notazione BNF)
  - **espressioni aritmetiche**: $e \in Exp$

    $$e := m \mid v \mid e + e \mid e - e \mid e * e$$
  - **espressioni booleane**: $b \in Bexp$

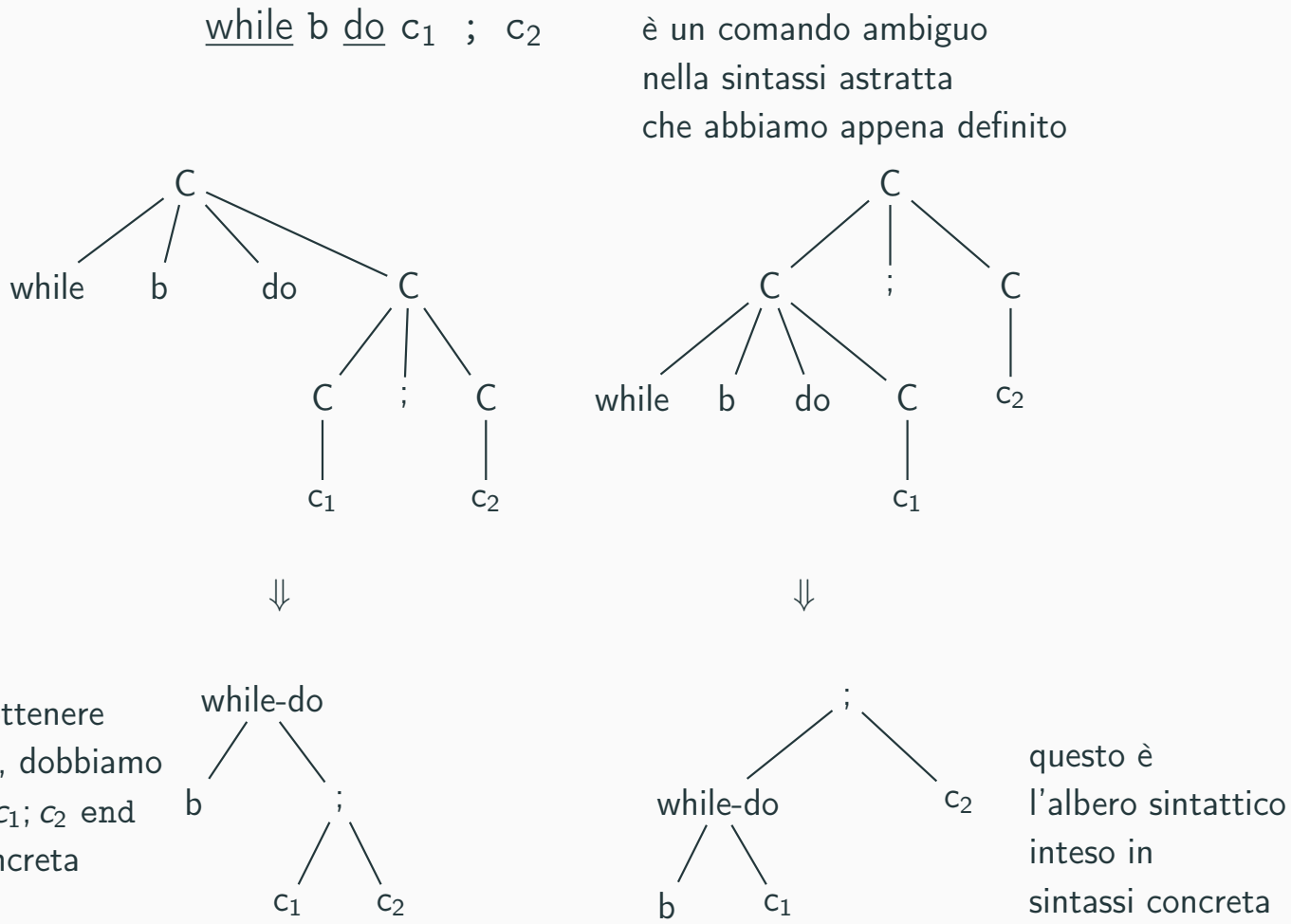
    $$b := t \mid e = e \mid b \ or \ b \mid \neg b$$
  - **comandi**: $c \in Com$

    $$c ::= skip \mid v := e \mid c; c \mid while \ b \ do \ c \mid if \ \ b \ then \ c \ else \ c$$

Sintassi concreta non ambigua molto più complicata, ma non serve nel dare semantica.

Perchè usiamo sintassi astratta?

Infatti, il parser prende un programma scritto in sintassi concreta e genera un albero sintattico di sintassi astratta. Quindi, nel dare semantica, possiamo partire dagli alberi di sintassi astratta, che non sono ambigui!



Quale modello usare?

Definiamo, per ogni categoria sintattica (cioè per *Exp*, *Bexp* e *Com*), un modello detto sistema di transizione.

Sistema di transizione
Un **sistema di transizione** è una tripla $< \Gamma, T, \rightarrow >$ dove

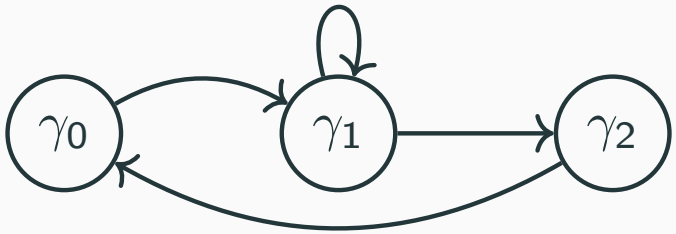
- Γ è l'insieme (possibilmente infinito) di stati (o configurazioni);
- $T \subseteq \Gamma$ è l'insieme degli stati terminali;
- $\rightarrow \subseteq \Gamma \times \Gamma$ è la relazione di transizione.

Una transizione $(\gamma_1, \gamma_2) \in \rightarrow$ viene denotata come $\gamma_1 \rightarrow \gamma_2$. Una computazione a partire dallo stato γ_0 è una sequenza di transizioni contigue $\gamma_0 \rightarrow \gamma_1 \rightarrow \gamma_2 \rightarrow \ldots$ che può essere finita o infinita.
Con $\rightarrow^*$ indichiamo la relazione di raggiungibilità, ovvero la chiusura riflessiva e transitiva di $\rightarrow$:

$$\frac{}{\gamma \rightarrow^* \gamma} \qquad \frac{\gamma \rightarrow^* \gamma' \quad \gamma' \rightarrow \gamma''}{\gamma \rightarrow^* \gamma''}$$

Esempio

Un sistema di transizione è rappresentato come un grafo dove i nodi sono gli stati e gli archi le transizioni. Ad esempio:



Tuttavia, noi dimostreremo che le relazioni di transizione per le espressioni aritmetiche, per le espressioni booleane e per i comandi sono tutte deterministiche, cioè da ogni stato parte al più una transizione, per cui la forma dei sistemi di transizione che andremo a definire sono del tipo:



Problemi

- Per dare significato alle variabili (che possono solo assumere valore su $\mathbb{N}$) è necessario introdurre uno **store** $\sigma : \mathit{Var} \to \mathbb{N}$, come funzione che associa ad ogni variabile un valore. Ad esempio:
$$\sigma = \{x_1/n_1, x_2/n_2, \ldots x_k/n_k\}$$
se supponiamo che $\mathit{Var} = \{x_1, x_2, \ldots, x_k\}$. Chiamiamo *Store* l'insieme di tutti i possibili store, cioè $\mathit{Store} = \{\sigma | \sigma : \mathit{Var} \to \mathbb{N}\}$
- Γ è di solito un insieme infinito contabile, quindi c'è necessità di trovare una rappresentazione finita implicita attraverso grammatiche/BNF. Cioè Γ essenzialmente coincide con uno dei linguaggi delle tre categorie sintattiche.
Ad esempio, $\Gamma_e = \{< e, \sigma > | e \in \mathit{Exp}, \sigma \in \mathit{Store}\}$ dove *Exp* è stato definito in stile BNF in precedenza.
- $\to \subseteq \Gamma \times \Gamma$ è una relazione costruita, tipicamente, da infinite coppie $(\gamma, \gamma') \in \to$, quindi c'è necessità di trovare una rappresentazione finita implicita come minima relazione che soddisfa un certo insieme finito di assiomi e regole di inferenza.

Semantica delle espressioni aritmetiche

È data dal sistema di transizione $< \Gamma_e, T_e, \rightarrow_e >$ dove:

- $\Gamma_e = \{< e, \sigma > | e \in Exp, \sigma \in Store\}$
- $T_e = \{< n, \sigma > | n \in \mathbb{N}, \sigma \in Store\}$
- la relazione $\rightarrow_e$ è definita come la minima relazione che soddisfa gli assiomi e le regole di inferenza seguenti (quelle della moltiplicazione, analoghe al quelle della somma, sono lasciate per esercizio):
  - nessuna regola/assioma per $< n, \sigma >$, il che significa che nessuna transizione parte da quelle configurazioni terminali.
  - un assioma per le variabili:

$$\frac{}{< v, \sigma > \rightarrow_e < \sigma(v), \sigma >} \ (Var)$$

Semantica delle espressioni aritmetiche (2)

$$\frac{< e_0, \sigma > \rightarrow_e < e_0', \sigma' >}{< e_0 + e_1, \sigma > \rightarrow_e < e_0' + e_1, \sigma' >} \ (Sum_1)$$

$$\frac{< e_1, \sigma > \rightarrow_e < e_1', \sigma' >}{< m + e_1, \sigma > \rightarrow_e < m + e_1', \sigma' >} \ (Sum_2)$$

$$\frac{m + m' = p}{< m + m', \sigma > \rightarrow_e < p, \sigma >} \ (Sum_3)$$

$$\frac{< e_0, \sigma > \rightarrow_e < e_0', \sigma' >}{< e_0 - e_1, \sigma > \rightarrow_e < e_0' - e_1, \sigma' >} \ (Sub_1)$$

$$\frac{< e_1, \sigma > \rightarrow_e < e_1', \sigma' >}{< m - e_1, \sigma > \rightarrow_e < m - e_1', \sigma' >} \ (Sub_2)$$

$$\frac{m \geq m' \quad p = m - m'}{< m - m', \sigma > \rightarrow_e < p, \sigma >} \ (Sub_3)$$

(Sum_3 e Sub_3 sono assiomi, dato che non ci sono transizioni come premesse, ma solo delle side-conditions)

Osservazioni

- Gli assiomi e le regole SOS sono facilmente implementabili in un linguaggio di programmazione logico, quale, ad esempio, Prolog, definendo così un semplice (ma inefficiente) interprete per le espressioni aritmetiche.
- Lo store σ non viene mai modificato durante la valutazione di una espressione. Si può dimostrare per induzione sulla prova della transizione $< e, \sigma > \rightarrow_e < e', \sigma' >$; infatti:
  - nessuno dei tre assiomi (Var, Sum_3, Sub_3) modifica lo store σ
  - se supponiamo che, per ogni regola, si abbia $\sigma = \sigma'$ nella premessa della regola, allora $\sigma = \sigma'$ anche nella conclusione. Quindi, σ non è mai modificato!
- L'assioma Sub_3 permette di derivare la transizione $< m - m', \sigma > \rightarrow_e < p, \sigma >$ solo se $m \geq m'$. Altrimenti $< m - m', \sigma >$ è bloccato pur non essendo uno stato terminale: è un caso di **"errore"** (perchè i numeri negativi non fanno parte dell'insieme di base del linguaggio, ma solo i naturali).

Osservazioni (2)

- Le regole Sum_1 e Sum_2 riflettono una disciplina (o regola) di valutazione detta IS (**Interna Sinistra**)
  - prima valutiamo l'argomento più a sinistra, ovvero e_0;
  - quando terminiamo la sua valutazione, si passa a valutare e_1;
  - quando termina anche quella, allora si fa la somma (Sum_3).
- Alternativamente, potremmo definire una disciplina ID (**Interna Destra**): si valuta prima e_1, poi e_0, infine si fa la somma. Le regole (Sum_1) e (Sum_2) vanno rimpiazzate dalle seguenti:

$$\frac{< e_1, \sigma > \rightarrow_e < e_1', \sigma' >}{< e_0 + e_1, \sigma > \rightarrow_e < e_0 + e_1', \sigma' >} \ (Sum_1')$$

$$\frac{< e_0, \sigma > \rightarrow_e < e_0', \sigma' >}{< e_0 + m, \sigma > \rightarrow_e < e_0' + m, \sigma' >} \ (Sum_2')$$

(Sum_3) rimane invariata.

Esempio

Vogliamo valutare l'espressione $(x + 2) - y$ nello store $\{x/5, y/3\}$.
Abbiamo i seguenti stati raggiungibili:

$$\gamma_0 = \langle (x+2) - y, \{x/5, y/3\} \rangle \quad \gamma_1 = \langle (5+2) - y, \{x/5, y/3\} \rangle$$
$$\gamma_2 = \langle 7 - y, \{x/5, y/3\} \rangle \quad \gamma_3 = \langle 7 - 3, \{x/5, y/3\} \rangle$$
$$\gamma_4 = \langle 4, \{x/5, y/3\} \rangle$$

$$\dfrac{\dfrac{\dfrac{}{\langle x, \{x/5, y/3\} \rangle \to_e \langle 5, \{x/5, y/3\} \rangle}\,\text{Var}}{\langle x+2, \{x/5, y/3\} \rangle \to_e \langle 5+2, \{x/5, y/3\} \rangle}\,\text{Sum}_1}{\gamma_0 = \langle (x+2) - y, \{x/5, y/3\} \rangle \to_e \langle (5+2) - y, \{x/5, y/3\} \rangle = \gamma_1}\,\text{Sub}_1$$

$$\dfrac{\dfrac{}{\langle 5+2, \{x/5, y/3\} \rangle \to_e \langle 7, \{x/5, y/3\} \rangle}\,\text{Sum}_3}{\gamma_1 = \langle (5+2) - y, \{x/5, y/3\} \rangle \to_e \langle 7 - y, \{x/5, y/3\} \rangle = \gamma_2}\,\text{Sub}_1$$

Esempio – continua

$$\gamma_0 = \langle (x+2) - y, \{x/5, y/3\} \rangle \quad \gamma_1 = \langle (5+2) - y, \{x/5, y/3\} \rangle$$
$$\gamma_2 = \langle 7 - y, \{x/5, y/3\} \rangle \quad \gamma_3 = \langle 7 - 3, \{x/5, y/3\} \rangle$$
$$\gamma_4 = \langle 4, \{x/5, y/3\} \rangle$$

$$\dfrac{\dfrac{}{\langle y, \{x/5, y/3\} \rangle \to_e \langle 3, \{x/5, y/3\} \rangle}\ \text{Var}}{\gamma_2 = \langle 7 - y, \{x/5, y/3\} \rangle \to_e \langle 7 - 3, \{x/5, y/3\} \rangle = \gamma_3}\ \text{Sub}_2$$

$$\dfrac{}{\gamma_3 = \langle 7 - 3, \{x/5, y/3\} \rangle \to_e \langle 4, \{x/5, y/3\} \rangle = \gamma_4}\ \text{Sub}_3$$

Abbiamo dimostrato che la computazione
$\gamma_0 \to_e \gamma_1 \to_e \gamma_2 \to_e \gamma_3 \to_e \gamma_4 \in T_e$ porta ad uno stato terminale; cioè $\langle (x+2) - y, \{x/5, y/3\} \rangle \to_e^* \langle 4, \{x/5, y/3\} \rangle$, ottenendo così il valore 4 dell'espressione $(x+2) - y$ nello store $\{x/5, y/3\}$.

Determinismo dimostrato per induzione strutturale

Vogliamo ora dimostrare che la relazione $\rightarrow_e$ è deterministica, ovvero che

$$\text{se } \gamma \rightarrow_e \gamma' \text{ e } \gamma \rightarrow_e \gamma'', \text{ allora } \gamma' = \gamma''.$$

Useremo una tecnica di prova che si chiama **Induzione Strutturale**.
Supponiamo di voler dimostrare una certa proprietà $P(e)$ vera per ogni $e \in Exp$ (ricordiamo che $e := m \mid v \mid e + e \mid e - e \mid e * e$).
Se dimostriamo che

1. $\forall m \in \mathbb{N}, P(m)$ è vera,
2. $\forall v \in Var, P(v)$ è vera,
3. se $P(e_0)$ e $P(e_1)$ sono vere, allora $P(e_0 + e_1)$ è vera,
4. se $P(e_0)$ e $P(e_1)$ sono vere, allora $P(e_0 - e_1)$ è vera,
5. se $P(e_0)$ e $P(e_1)$ sono vere, allora $P(e_0 * e_1)$ è vera,

allora possiamo concludere che $\forall e \in Exp, P(e)$ è vera.
Nel nostro caso, $P(e)$ è la proprietà

$$\forall \sigma, \gamma', \gamma'' : (< e, \sigma > \rightarrow_e \gamma' \wedge < e, \sigma > \rightarrow_e \gamma'') \Rightarrow \gamma' = \gamma''$$

Dimostrazione per induzione strutturale

Dimostriamo che $P(e)$ è vera $\forall e \in Exp$ usando induzione strutturale.

1. Caso $e = m \in \mathbb{N}$: osserva che $< m, \sigma > \not\rightarrow$ e allora $P(m)$ è vera perchè la premessa dell'implicazione è falsa (caso "vacuo").
2. Caso $e = v \in Var$: per la regola (Var), l'unica transizione derivabile per $< v, \sigma >$ è $< v, \sigma > \rightarrow_e < \sigma(v), \sigma >$. Poiché σ è una funzione (cioè $\sigma(v)$ è univoco), la tesi segue:
   $(< v, \sigma > \rightarrow_e \gamma' \wedge\ < v, \sigma > \rightarrow_e \gamma'') \ \Rightarrow\ \gamma' = \gamma''$
3. Caso $e = e_0 + e_1$: supponiamo che $< e_0 + e_1, \sigma > \rightarrow_e \gamma'$ e $< e_0 + e_1, \sigma > \rightarrow_e \gamma''$. Ci sono tre sotto-casi da esaminare in accordo al modo in cui derivo $< e_0 + e_1, \sigma > \rightarrow_e \gamma'$

Dimostrazione per induzione strutturale (2)

- Sottocaso $< e_0, \sigma > \rightarrow < e_0', \sigma' >$ e $\gamma' = < e_0' + e_1, \sigma' >$:
  In questo caso, $e_0 \notin \mathbb{N}$ e l'unica regola applicabile è (Sum_1). Allora, se $< e_0 + e_1, \sigma > \rightarrow_e \gamma''$, è necessario che $< e_0, \sigma > \rightarrow_e < e_0'', \sigma'' >$ e $\gamma'' = < e_0'' + e_1, \sigma'' >$. Ma, per ipotesi induttiva, $P(e_0)$ vale, per cui deve essere $e_0' = e_0''$ e $\sigma' = \sigma''$, da cui discende $\gamma' = \gamma''$.
- Sottocaso $e_0 = m \in \mathbb{N}$ ed $< e_1, \sigma > \rightarrow_e < e_1', \sigma' >$:
  In questo caso, $e_1 \notin \mathbb{N}$ e l'unica regola applicabile è (Sum_2). Allora, se $< e_0 + e_1, \sigma > \rightarrow_e \gamma''$, è necessario che $< e_1, \sigma > \rightarrow_e < e_1'', \sigma'' >$ e $\gamma'' = < e_0 + e_1'', \sigma'' >$. Ma, per ipotesi induttiva, $P(e_1)$ vale, per cui deve essere $e_1' = e_1''$ e $\sigma' = \sigma''$, da cui discende $\gamma' = \gamma''$.
- Sottocaso $e_0 \in \mathbb{N}$ ed $e_1 \in \mathbb{N}$:
  In questo caso, solo (Sum_3) è applicabile, ottenendo una sola possibile transizione $< e_0 + e_1, \sigma > \rightarrow_e < p, \sigma >$ dove $p = e_0 + e_1$.

Quindi la tesi segue:
$(< e_0 + e_1, \sigma > \rightarrow \gamma' \wedge < e_0 + e_1, \sigma > \rightarrow \gamma'') \Rightarrow \gamma' = \gamma''$.
Gli altri due casi $e = e_1 - e_2$ ed $e = e_1 * e_2$ sono analoghi al caso sopra, fateli per esercizio. *Fine della prova.*

Funzione eval

Poiché $\rightarrow_e$ è deterministica, a partire da $< e, \sigma >$ arriveremo su un solo stato terminale $< n, \sigma >$: quindi "n è il valore di e in σ".
È possibile perciò definire una funzione parziale

$$\mathit{eval} : \mathit{Exp} \times \mathit{Store} \rightarrow\!\!\!\!\circ\; \mathbb{N}$$

$$\mathit{eval}(e, \sigma) = \begin{cases} m & \text{se } < e, \sigma > \rightarrow_e^* < m, \sigma > \\ \text{indefinita} & \text{altrimenti} \end{cases}$$

- Ad esempio, $\mathit{eval}((x+2)-y, \{x/5, y/3\}) = 4$ perché $< (x+2)-y, \{x/5, y/3\} > \rightarrow_e^* < 4, \{x/5, y/3\} >$.
- La funzione *eval* è parziale perché non sempre si raggiunge uno stato terminale:
  - ad esempio, $\mathit{eval}((x+2)-y, \{x/2, y/7\}) =$ indefinito perché $< (x+2)-y, \{x/2, y/7\} > \rightarrow_e^* < 4-7, \{x/2, y/7\} > \not\rightarrow_e$, cioè lo stato raggiunto è bloccato (deadlock), ma non terminale.

Esercizi ed equivalenza tra espressioni

Esercizi

- Provate a calcolare $eval((2 * x) + y, \{x/2, y/7\})$, dimostrando ogni singola transizione della computazione che porta sullo stato terminale $< 11, \{x/2, y/7\} >$.
- Dimostrate che per ogni $e \in Exp$, e per ogni $\sigma \in Store$, esiste una espressione $e' \in Exp$ tale che $< e, \sigma > \rightarrow_e^* < e', \sigma > \not\rightarrow_e$, cioè la valutazione di una espressione non diverge.

Equivalenza tra espressioni

Due espressioni $e, e' \in Exp$ sono equivalenti, e lo denotiamo con $e \equiv e'$, se e solo se

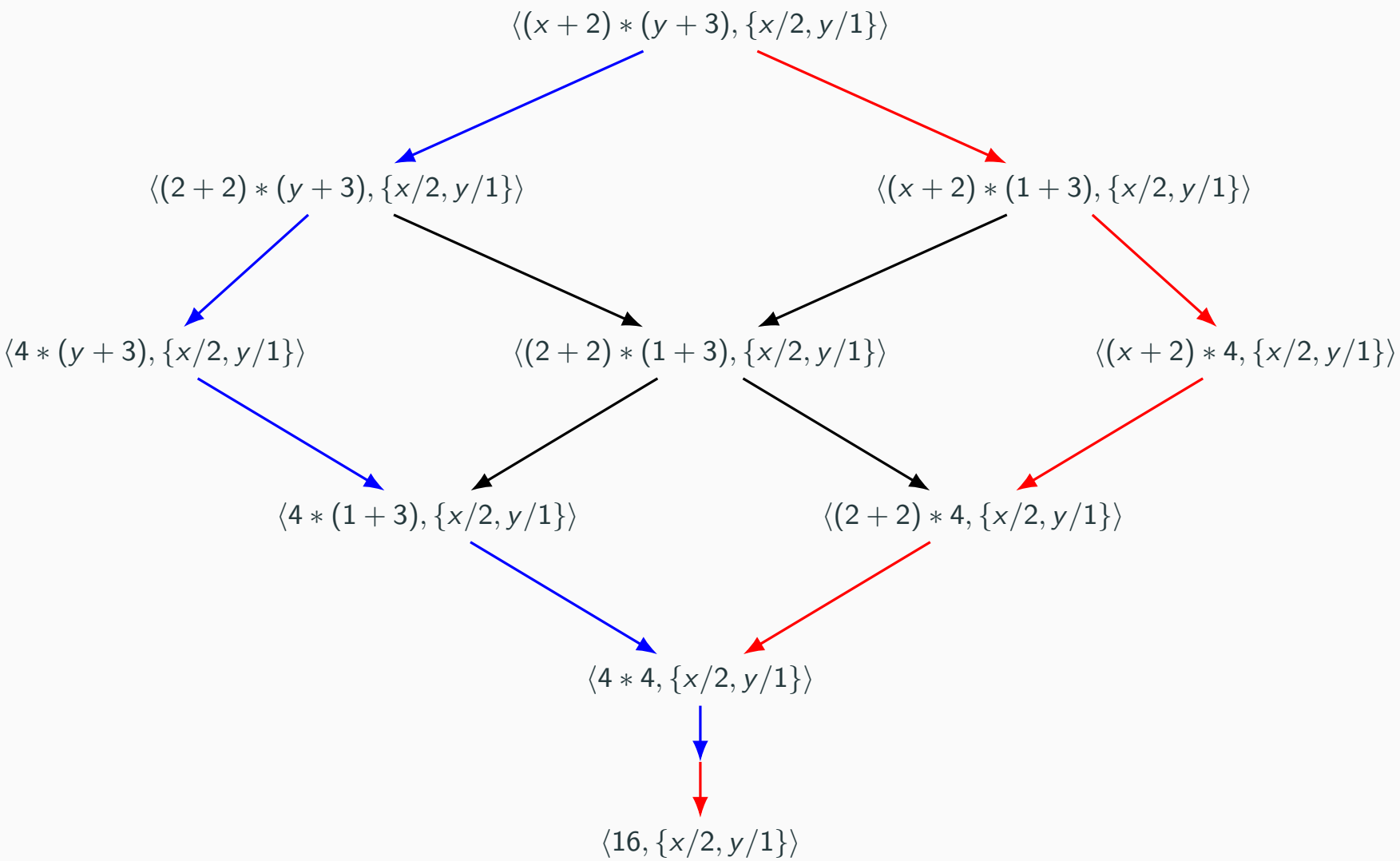
$$eval(e, \sigma) = eval(e', \sigma) \qquad \forall \sigma \in Store$$

Ad esempio, $v_1 + (v_2 + v_3) \equiv (v_1 + v_2) + v_3$

Osservazioni

- La funzione parziale *eval* è definita rispetto alla disciplina di valutazione IS. A rigore, dovremmo chiamarla $eval_{IS}$.
- Si può dimostrare che anche per la disciplina ID, il risultato della valutazione è lo stesso (perché?): $eval_{IS} = eval_{ID}$.
- Come vedremo, è possibile definire anche altre discipline di valutazione, come ES, ED, EP (E=esterna, S= sinistra, D=destra, P=parallela) ma queste sono diverse da IS=ID.
- Ad esempio, la regola IP (interna-parallela) prevede che le regole SOS per la somma siano (Sum_1) da IS, (Sum'_1) da ID e (Sum_3).
  - La relazione $\rightarrow_{IP}$ risulta nondeterministica, ma confluente, cioè termina in modo univoco.
  - Si può dimostrare che $eval_{IP} = eval_{IS}$ (perché?)

Confronto tra IP, IS e ID



Semantica delle espressioni booleane

Espressioni booleane: $b \in Bexp$

$$b := t \mid e = e \mid b \text{ or } b \mid \neg b$$

La loro semantica è data dal sistema di transizione $< \Gamma_b, T_b, \rightarrow_b >$ dove:

- $\Gamma_b = \{< b, \sigma > | b \in Bexp, \sigma \in Store\}$
- $T_b = \{< tt, \sigma >, < ff, \sigma > | \sigma \in Store\}$
- la relazione di transizione $\rightarrow_b$ è definita come la minima relazione che soddisfa gli assiomi e le regole di inferenza seguenti.

$$\frac{< e_0, \sigma > \rightarrow_e < e_0', \sigma' >}{< e_0 = e_1, \sigma > \rightarrow_b < e_0' = e_1, \sigma' >} \ (Eq_1)$$

$$\frac{< e_1, \sigma > \rightarrow_e < e_1', \sigma' >}{< m = e_1, \sigma > \rightarrow_b < m = e_1', \sigma' >} \ (Eq_2)$$

$$\frac{t = tt \text{ se } m = n, t = ff \text{ altrimenti}}{< m = n, \sigma > \rightarrow_b < t, \sigma >} \ (Eq_3)$$

Nota che l'espressione booleana $e_0 = e_1$ viene valutata con disciplina IS.

Semantica delle espressioni booleane (2)

Valutazione ES (esterna-sinistra) per l'espressione b_0 *or* b_1: non valuto sempre tutti e due gli argomenti!

$$\frac{< b_0, \sigma > \rightarrow_b < b_0', \sigma' >}{< b_0 \text{ or } b_1, \sigma > \rightarrow_b < b_0' \text{ or } b_1, \sigma' >} \ (Or_1)$$

$$\frac{}{< tt \text{ or } b_1, \sigma > \rightarrow_b < tt, \sigma >} \ (Or_2)$$

$$\frac{}{< ff \text{ or } b_1, \sigma > \rightarrow_b < b_1, \sigma >} \ (Or_3)$$

$$\frac{< b, \sigma > \rightarrow_b < b', \sigma' >}{< \neg b, \sigma > \rightarrow_b < \neg b', \sigma' >} \ (Neg_1)$$

$$\frac{t' = tt \text{ se } t = ff,\ t' = ff \text{ se } t = tt}{< \neg t, \sigma > \rightarrow_b < t', \sigma >} \ (Neg_2)$$

Osservazioni

- Lo store σ non viene mai modificato durante la valutazione di una espressione booleana (si dimostra che, se $< b, \sigma > \rightarrow_b < b', \sigma' >$, allora $\sigma = \sigma'$ per induzione sulla prova della transizione):
  - nessuno degli assiomi (Eq_3, Or_2, Or_3, Neg_2) modifica lo store σ,
  - per le regole Eq_1 e Eq_2, sappiamo già che le transizioni nella premessa hanno $\sigma = \sigma'$, quindi anche nella conclusione $\sigma = \sigma'$,
  - per le regole Or_1 e Neg_1, se supponiamo che le transizioni nella premessa hanno $\sigma = \sigma'$, allora anche nella conclusione $\sigma = \sigma'$.
- la relazione di transizione $\rightarrow_b$ è deterministica, ovvero se $\gamma \rightarrow_b \gamma'$ e $\gamma \rightarrow_b \gamma''$, allora $\gamma' = \gamma''$ (si può dimostrare come fatto per $\rightarrow_e$).
  Per cui, si può facilmente definire

$$eval_B(b, \sigma) = \begin{cases} t & \text{se } < b, \sigma > \rightarrow_b^* < t, \sigma > \\ \text{indefinita} & \text{altrimenti} \end{cases}$$

  e l'equivalenza

$$b \equiv b' \text{ sse } eval_B(b, \sigma) = eval_B(b', \sigma) \quad \forall \sigma \in Store$$

Semantiche alternative per le espressioni booleane

Si possono definire per b_0 *or* b_1 regole di valutazione diverse da ES, ad esempio ED:

$$\frac{< b_1, \sigma > \rightarrow_b < b_1', \sigma' >}{< b_0 \text{ or } b_1, \sigma > \rightarrow_b < b_0 \text{ or } b_1', \sigma' >} \ (Or_1')$$

$$\frac{}{< b_0 \text{ or } tt, \sigma > \rightarrow_b < tt, \sigma >} \ (Or_2')$$

$$\frac{}{< b_0 \text{ or } ff, \sigma > \rightarrow_b < b_0, \sigma >} \ (Or_3')$$

ma non sono equivalenti. Ad esempio, per $\gamma =< (3-5) = 6 \text{ or } tt, \sigma >$:

$\gamma \not\rightarrow_{ES}$ perchè la valutazione della sotto-espressione 3-5 è bloccata

$\gamma \rightarrow_{ED} < tt, \sigma >$

In generale:

$$eval_{B_{ES}} \neq eval_{B_{ED}} \neq eval_{B_{IS}} = eval_{B_{ID}}$$

Esercizi

- Definire le regole IS per la valutazione dell'espressione b_0 *or* b_1
  - mostrare un esempio per il quale la valutazione IS e quella ES non forniscono lo stesso risultato
- Definire le regole ID per la valutazione dell'espressione b_0 *or* b_1
  - mostrare un esempio per il quale la valutazione ID e quella ED non forniscono lo stesso risultato
- Definire le regole EP (esterna-parallela) per b_0 *or* b_1 (Suggerimento: ricorda che una regola parallela non fissa nessuna priorità sull'ordine di valutazione dei due argomenti.)
  - Mostrare un esempio per il quale la valutazione ES e quella EP non forniscono lo stesso risultato.
  - Mostrare un esempio per il quale la valutazione ED e quella EP non forniscono lo stesso risultato.

Osservazioni

In generale, si può affermare che:

- se, prese due discipline di valutazione diverse (ad esempio IS ed ES), la valutazione di una espressione termina per entrambe, allora il risultato è sempre lo stesso;
- se una disciplina interna fornisce un risultato, anche una disciplina esterna lo darà;
- le discipline di valutazione esterne a volte danno un risultato quando una interna non è in grado di darlo (perché si blocca);
- la disciplina di valutazione EP è quella più definita di tutte.

Infine, nei normali linguaggi di programmazione, la valutazione delle espressioni aritmetiche viene comunemente fatta con disciplina IS, mentre la valutazione delle espressioni booleane viene fatta con disciplina ES (entrambe deterministiche!)

Esempio: prodotto con ES

$$\frac{}{< 0 * e_1, \sigma > \rightarrow_e < 0, \sigma >} \ (P_1)$$

$$\frac{}{< 1 * e_1, \sigma > \rightarrow_e < e_1, \sigma >} \ (P_2)$$

$$\frac{< e_0, \sigma > \rightarrow_e < e_0', \sigma' >}{< e_0 * e_1, \sigma > \rightarrow_e < e_0' * e_1, \sigma' >} \ (P_3)$$

$$\frac{< e_1, \sigma > \rightarrow_e < e_1', \sigma' > \quad m \neq 0, 1}{< m * e_1, \sigma > \rightarrow_e < m * e_1', \sigma' >} \ (P_4)$$

$$\frac{m \neq 0, 1 \text{ and } p = m * n}{< m * n, \sigma > \rightarrow_e < p, \sigma' >} \ (P_5)$$

Nota che IS e ES sono differenti. Ad esempio per $\gamma =< 0 * (3 - 5), \sigma >$:

$\gamma \not\rightarrow_{IS}$ perchè 3-5 è bloccato

$\gamma \rightarrow_{ES} < 0, \sigma >$

Semantica dei comandi

Comandi: $c \in Com$

$$c := skip \mid v := e \mid c; c \mid while\ b\ do\ c \mid if\ b\ then\ c\ else\ c$$

La loro semantica è data dal sistema di transizione $< \Gamma_c, T_c, \rightarrow_c >$ dove:

- $\Gamma_c = \{< c, \sigma > | c \in Com, \sigma \in Store\} \cup \{\sigma | \sigma \in Store\}$
- $T_c = \{\sigma | \sigma \in Store\} = Store$
- la relazione di transizione $\rightarrow_c \subseteq \Gamma_c \times \Gamma_c$ è definita come la minima relazione che soddisfa gli assiomi e le regole di inferenza seguenti:

$$\frac{}{< skip, \sigma > \rightarrow_c \sigma} \ (Skip)$$

$$\frac{< e, \sigma > \rightarrow_e^* < m, \sigma >}{< v := e, \sigma > \rightarrow_c \sigma[m/v]} \ (Ass)$$

$$\text{dove} \quad \sigma[m/v](v') = \begin{cases} m & \text{se } v = v' \\ \sigma(v') & \text{altrimenti} \end{cases}$$

è l'operazione di aggiornamento dello store σ: ad esempio, se $\sigma = \{x/1, y/1\}$, allora $\sigma[3/x] = \{x/3, y/1\}$.

Semantica dei comandi (2)

$$\frac{< c_0, \sigma > \rightarrow_c < c_0', \sigma' >}{< c_0; c_1, \sigma > \rightarrow_c < c_0'; c_1, \sigma' >} \ (Seq_1)$$

$$\frac{< c_0, \sigma > \rightarrow_c \sigma'}{< c_0; c_1, \sigma > \rightarrow_c < c_1, \sigma' >} \ (Seq_2)$$

$$\frac{< b, \sigma > \rightarrow_b^* < tt, \sigma >}{< \mathit{if}\ \ b\ \mathit{then}\ c_0\ \mathit{else}\ c_1, \sigma > \rightarrow_c < c_0, \sigma >} \ (If_1)$$

$$\frac{< b, \sigma > \rightarrow_b^* < ff, \sigma >}{< \mathit{if}\ \ b\ \mathit{then}\ c_0\ \mathit{else}\ c_1, \sigma > \rightarrow_c < c_1, \sigma >} \ (If_2)$$

Semantica dei comandi (3)

$$\frac{< b, \sigma > \rightarrow_b^* < tt, \sigma >}{< \textit{while } b \textit{ do } c, \sigma > \rightarrow_c < c; \textit{while } b \textit{ do } c, \sigma >} \ (While_1)$$

$$\frac{< b, \sigma > \rightarrow_b^* < ff, \sigma >}{< \textit{while } b \textit{ do } c, \sigma > \rightarrow_c \sigma} \ (While_2)$$

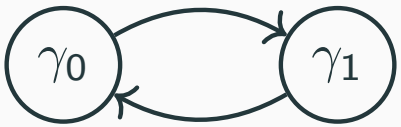
Oppure

$$\frac{}{< \textit{while } b \textit{ do } c, \sigma > \rightarrow_c < \textit{if } b \textit{ then } c; (\textit{while } b \textit{ do } c) \ \textit{ else skip}, \sigma >} \ (Alt - W)$$

Esempio (1): sistema divergente "ciclico"

$$\frac{\langle tt, \sigma\rangle \to_b^* \langle tt, \sigma\rangle}{\gamma_0 = \langle \mathit{while}\ tt\ \mathit{do}\ \mathit{skip}, \sigma\rangle \to_c \langle \mathit{skip}; (\mathit{while}\ tt\ \mathit{do}\ \mathit{skip}), \sigma\rangle = \gamma_1}\ \textsf{While}_1$$

$$\frac{\dfrac{}{\langle \mathit{skip}, \sigma\rangle \to_c \sigma}\ \textsf{Skip}}{\gamma_1 = \langle \mathit{skip}; (\mathit{while}\ tt\ \mathit{do}\ \mathit{skip}), \sigma\rangle \to_c \langle \mathit{while}\ tt\ \mathit{do}\ \mathit{skip}, \sigma\rangle = \gamma_0}\ \textsf{Seq}_2$$

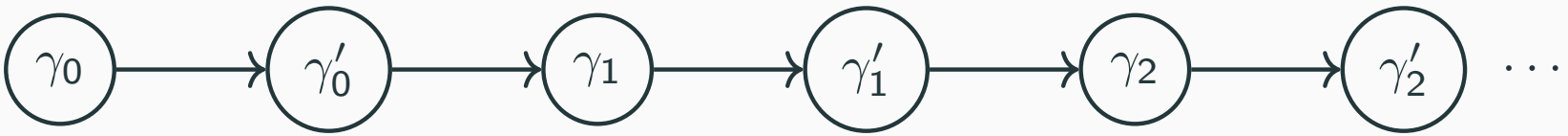


Esempio (2): sistema divergente "infinito"

$\gamma_i = \langle \mathit{while}\ tt\ \mathit{do}\ x := x + 1, \{x/i\} \rangle$
$\gamma_i' = \langle x := x + 1; (\mathit{while}\ tt\ \mathit{do}\ x := x + 1), \{x/i\} \rangle$

$$\frac{\langle tt, \{x/0\} \rangle \to_b^* \langle tt, \{x/0\} \rangle}{\gamma_0 \to_c \gamma_0'}\ \text{While}_1$$

$$\cfrac{\cfrac{\cfrac{}{\langle x, \{x/0\} \rangle \to_e \langle 0, \{x/0\} \rangle}\ \text{Var}}{\langle x + 1, \{x/0\} \rangle \to_e \langle 0 + 1, \{x/0\} \rangle \to_e \langle 1, \{x/0\} \rangle}\ \text{Sum}_1}{\langle x := x + 1, \{x/0\} \rangle \to_e \{x/1\}}\ \text{Ass}$$
$$\frac{\langle x := x + 1, \{x/0\} \rangle \to_e \{x/1\}}{\gamma_0' \to_c \gamma_1}\ \text{Seq}_2$$



Funzione exec

La relazione di transizione $\rightarrow_c$ è deterministica, cioè se $\gamma \rightarrow_c \gamma'$ e $\gamma \rightarrow_c \gamma''$, allora $\gamma' = \gamma''$. È quindi possibile definire la funzione parziale

$$exec : Com \times Store \multimap Store$$

$$exec(c, \sigma) = \begin{cases} \sigma' & \text{se } < c, \sigma > \rightarrow_c^* \sigma' \\ \text{indefinita} & \text{altrimenti} \end{cases}$$

- Osserva che *exec* è parziale perché
  - un comando può non terminare (divergenza): $c_1 = \mathit{while\ tt\ do\ skip}$ $< c_1, \sigma > \rightarrow_c < \mathit{skip}\ ;\ c_1, \sigma > \rightarrow_c < c_1, \sigma > \rightarrow_c < \mathit{skip}\ ;\ c_1, \sigma > \rightarrow_c \ldots$
  - ma anche perché la valutazione di una espressione aritmetica può bloccarsi (deadlock): $c_2 = x := (3 - 5)$ $< c_2, \sigma > \not\rightarrow$
- Quindi $exec(c_1, \sigma) = \text{indefinito} = exec(c_2, \sigma)$. Se vogliamo semanticamente distinguere questi aspetti computazionali, dobbiamo raffinare la semantica, introducendo più dettagli.

Errori dinamici (a runtime)

È necessario estendere l'insieme degli stati con uno stato apposito (err) di segnalazione dell'errore.

$$\Gamma' = \Gamma \cup \{err\}$$

$$T' = T \cup \{err\}$$

$$eval : Exp \times Store \to \mathbb{N} \cup \{err\} \quad \text{(ora è totale!)}$$

$$eval_B : Bexp \times Store \to \mathbb{T} \cup \{err\} \quad \text{(ora è totale!)}$$

$$exec : Com \times Store \multimap Store \cup \{err\}$$

(rimane parziale per modellare la divergenza)

Unica regola per **generare** "errore":

$$\frac{m' > m}{< m - m', \sigma > \to err} \ (Sub_4)$$

Questo è l'unico errore possibile nel nostro semplice linguaggio: sottrazione con risultato negativo (underflow).

Regole per propagare l'errore

In *Exp* (queste per $+$, analoghe per $-$ e $*$):

$$\frac{< e_0, \sigma > \rightarrow_e err}{< e_0 + e_1, \sigma > \rightarrow_e err}\ (Sum_4) \qquad \frac{< e_1, \sigma > \rightarrow_e err}{< m + e_1, \sigma > \rightarrow_e err}\ (Sum_5)$$

In *Bexp* (oltre ad analoghe a quelle sopra per $e_0 = e_1$ (Eq_4 e Eq_5)):

$$\frac{< b, \sigma > \rightarrow_b err}{< \neg b, \sigma > \rightarrow_b err}\ (Neg_3) \qquad \frac{< b_0, \sigma > \rightarrow_b err}{< b_0\ or\ b_1, \sigma > \rightarrow_b err}\ (Or_4)$$

In *Com*:

$$\frac{< e, \sigma > \rightarrow_e^* err}{< v := e, \sigma > \rightarrow_c err}\ (Ass_2) \qquad \frac{< c_0, \sigma > \rightarrow_c err}{< c_0; c_1, \sigma > \rightarrow_c err}\ (Seq_3)$$

$$\frac{< b, \sigma > \rightarrow_b^* err}{< if\ b\ then, c_0\ else\ c_1, \sigma > \rightarrow_c err}\ (If_3) \qquad \frac{< b, \sigma > \rightarrow_b^* err}{< while\ b\ do\ c, \sigma > \rightarrow_c err}\ (While_3)$$

Tutte queste regole per un solo tipo di errore dinamico: underflow generato da (Sub_4). Altri errori dinamici possibili in un linguaggio più esteso: divisione per zero (se abbiamo divisione), overflow (se fissiamo un maxinteger), index out of bounds (se introduciamo gli array),... Servono tanti stati di errore distinti e ... tantissime regole!

Nondeterminismo e parallelismo

Aggiungiamo due nuovi comandi, **or** e **par**

$$c ::= \ skip \mid v := e... \mid c \ \textbf{or} \ c \ \mid c \ \textbf{par} \ c$$

le cui regole sono:

$$\frac{}{< c_0 \ \textbf{or} \ c_1, \sigma > \rightarrow_c < c_0, \sigma >} \ (Nd_1) \qquad \frac{}{< c_0 \ \textbf{or} \ c_1, \sigma > \rightarrow_c < c_1, \sigma >} \ (Nd_2)$$

$$\frac{< c_0, \sigma > \rightarrow_c < c_0', \sigma' >}{< c_0 \ \textbf{par} \ c_1, \sigma > \rightarrow_c < c_0' \ \textbf{par} \ c_1, \sigma' >} \ (Par_1)$$

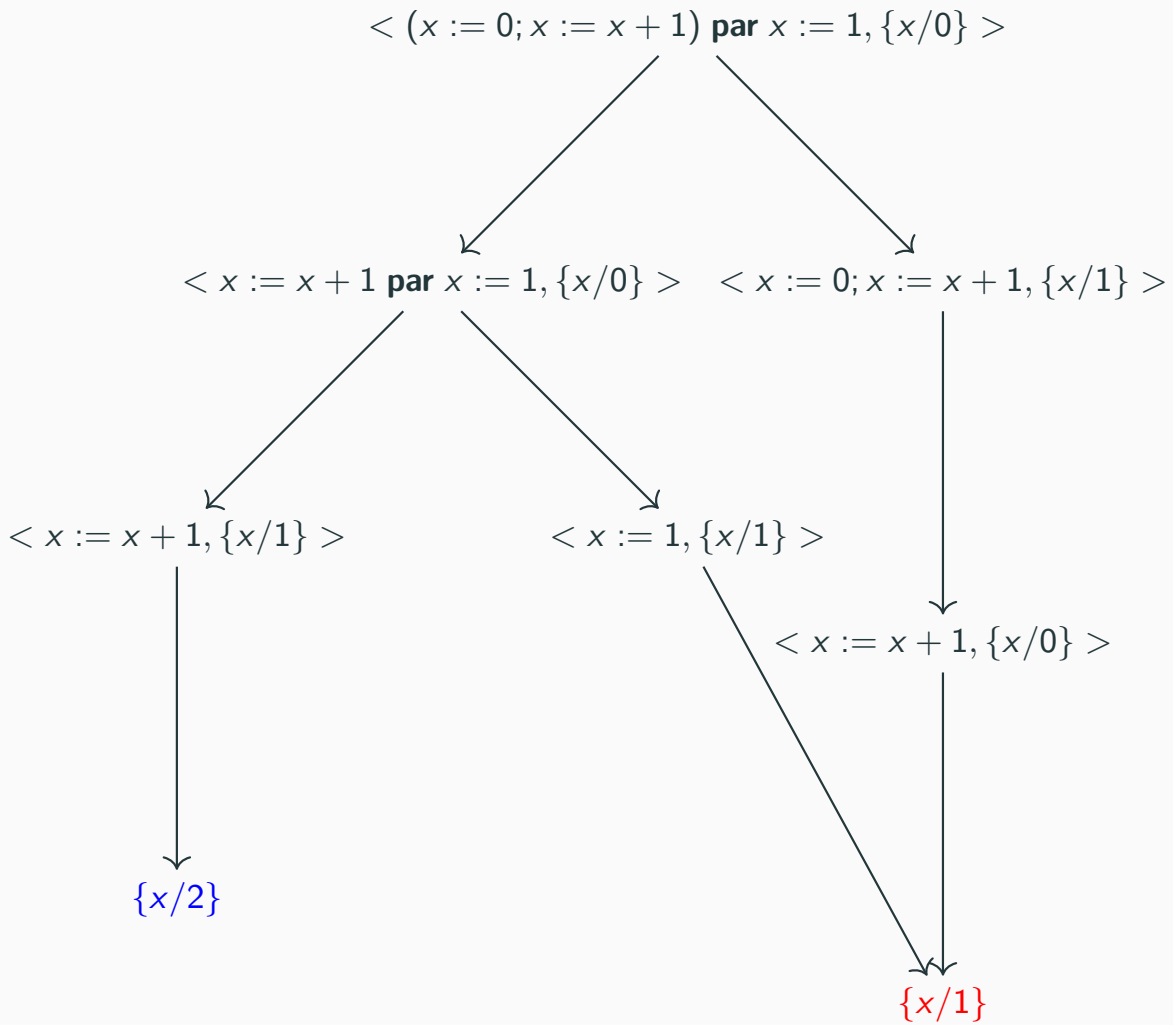
$$\frac{< c_0, \sigma > \rightarrow_c \sigma'}{< c_0 \ \textbf{par} \ c_1, \sigma > \rightarrow_c < c_1, \sigma' >} \ (Par_2)$$

$$\frac{< c_1, \sigma > \rightarrow_c < c_1', \sigma' >}{< c_0 \ \textbf{par} \ c_1, \sigma > \rightarrow_c < c_0 \ \textbf{par} \ c_1', \sigma' >} \ (Par_3)$$

$$\frac{< c_1, \sigma > \rightarrow_c \sigma'}{< c_0 \ \textbf{par} \ c_1, \sigma > \rightarrow_c < c_0, \sigma' >} \ (Par_4)$$

Nondeterminismo e parallelismo (2)

La funzione $\rightarrow_c$ non è più deterministica sul linguaggio esteso con **or** e **par**. Per verificarlo, costruiamo il sistema di transizione per $(x := 0; x := x + 1)$ **par** $x := 1$ con store $\{x/0\}$.



Nondeterminismo e parallelismo (3)

Nell'esempio precedente ci sono due possibili risultati finali: $\{x/1\}$ o $\{x/2\}$. Per modellare questa situazione, possiamo pensare di modificare la funzione *exec* per restituire non uno store, ma un insieme di store:

$$exec : Com \times Store \multimap \mathcal{P}(Store)$$

Ad esempio

$$exec((x := 0; x := x + 1) \textbf{ par } x := 1, \{x/0\}) = \{\{x/1\}, \{x/2\}\}$$

Ma questa soluzione è piuttosto approssimativa. Ad esempio, se consideriamo $c_1 = \mathit{while\ tt\ do\ skip}$, allora

$$exec(x := 3 \textbf{ or } c_1, \sigma) = exec(x := 3, \sigma) = \{\sigma[3/x]\}$$

il che non è davvero adeguato, perché farebbe pensare che l'esecuzione di $x := 3 \textbf{ or } c_1$ sia deterministica, producendo sempre come risultato $\sigma[3/x]$. (Tecniche adeguate per risolvere questo problema sono al di là dello scopo di questa lezione introduttiva.)

Nondeterminismo e parallelismo (4)

Per quanto riguarda la gestione dell'errore dinamico (underflow), è necessario aggiungere due regole (solo per **par**, nessuna per **or**):

$$\frac{< c_0, \sigma > \rightarrow_c err}{< c_0 \textbf{ par } c_1, \sigma > \rightarrow_c err} \qquad \frac{< c_1, \sigma > \rightarrow_c err}{< c_0 \textbf{ par } c_1, \sigma > \rightarrow_c err}$$